Python Programming

A Beginner's Reference

DCU International Foundation Year

Computer Science

A quick reference for writing Python 3 programs.

1 Fundamentals

1.1 Variables and basic types

Python	Meaning or result
`name = "Alex"`	str: text inside quotes.
`age = 18`	int: a whole number.
`price = 2.50`	float: a decimal number.
`finished = False`	bool: True or False, with capital letters.
`result = None`	None: no value yet.
`score += 1`	Add 1 to score.
`score -= 1`	Subtract 1 from score.
`score *= 2`	Multiply score by 2.
`score /= 2`	Divide score by 2.
`# This is a comment`	A comment explains code and is not executed.

Names are case-sensitive: score and Score are different. Use names such as total_cost; avoid spaces, keywords, and names of built-in functions such as list or str.

1.2 Input output and conversion

Python	Meaning or result
`print("Hello", name)`	Display text and values.
`name = input("Name:  ")`	Read keyboard input. The result is always a string.
`age = int(input("Age:  "))`	Convert typed text to a whole number.
`price = float(input("Price:  "))`	Convert typed text to a decimal number.
`str(42)`	Convert to the string "42".
`int("42")`	Convert to the integer 42.
`float("3.5")`	Convert to the decimal number 3.5.
`type(3.5)`	Check the type: float.
`print(f"Hi {name}, age {age}")`	Insert variable values into an f-string.
`print(f"€{price:.2f}")`	Display a number with two decimal places.
`print("A\nB")`	\n starts a new line.
`print("A\tB")`	\t inserts a tab.

1.3 Arithmetic operations

Operator	Meaning	Example	Result
`+`	Addition	`7 + 2`	`9`
`-`	Subtraction	`7 - 2`	`5`
`*`	Multiplication	`7 * 2`	`14`
`/`	Division	`7 / 2`	`3.5`
`//`	Floor division	`7 // 2`	`3`
`%`	Remainder (modulo)	`7 % 2`	`1`
`**`	Exponentiation (power)	`2 ** 3`	`8`
`( )`	Parentheses: evaluate first	`(2 + 3) * 4`	`20`
`abs()`	Absolute value	`abs(-8)`	`8`
`round()`	Round to two decimal places	`round(3.14159, 2)`	`3.14`

Order: parentheses, powers, multiplication/division/remainder, then addition/subtraction. // rounds down: -7 // 2 is -4. Use ** for powers, not ^.

2 Decisions loops and functions

2.1 Comparisons and logic

Comparison operators

Operator	Meaning	Example	Result
`==`	Equal to	`5 == 5`	`True`
`!=`	Not equal to	`5 != 5`	`False`
`<`	Less than	`3 < 5`	`True`
`<=`	Less than or equal to	`5 <= 5`	`True`
`>`	Greater than	`3 > 5`	`False`
`>=`	Greater than or equal to	`5 >= 5`	`True`

Boolean operators

Operator	Meaning	Example	Result
`and`	True only when both conditions are true	`True and False`	`False`
`or`	True when at least one condition is true	`True or False`	`True`
`not`	Reverse the Boolean value	`not True`	`False`

Membership operators

Operator	Meaning	Example	Result
`in`	True when a value is present	`"a" in "cat"`	`True`
`not in`	True when a value is absent	`"z" not in "cat"`	`True`

2.2 Choose with if elif and else

```
mark = 65
if mark >= 70:
    print("Excellent")
elif mark >= 40:
    print("Pass")
else:
    print("Try again")
```

Python runs the first matching branch. Use a colon after each condition and else. Indent the body consistently with four spaces.

2.3 Repeat with for and range

```
for number in range(1, 4):
    print(number)          # 1, 2, 3
```

Python	Meaning or result
`range(5)`	0, 1, 2, 3, 4
`range(2, 6)`	2, 3, 4, 5
`range(0, 10, 2)`	0, 2, 4, 6, 8
`range(3, 0, -1)`	3, 2, 1. The stop value is always excluded.

2.4 Repeat with while

```
count = 3
while count > 0:
    print(count)
    count -= 1             # Change the condition to avoid an endless loop
```

break exits the nearest loop. continue skips the rest of the current iteration and starts the next one.

2.5 Define and call a function

```
def area(width, height):
    return width * height
answer = area(4, 3)        # Call the function; answer is 12
print(answer)
```

Parameters receive inputs. return sends a value back to the caller; print only displays it. A function without an explicit return value returns None.

3 Strings and lists

3.1 Working with text

For these examples, use text = "Python". Indexes start at 0. Negative indexes count backwards from the end.

Python	Meaning or result
`len(text)`	6: number of characters.
`text[0]`	"P": the first character.
`text[-1]`	"n": the last character.
`text[1:4]`	"yth": start included, stop excluded.
`text[:3]`	"Pyt": slice from the beginning to index 3, excluded.
`text[3:]`	"hon": slice from index 3 to the end.
`text.lower()`	Return lowercase text: "python".
`text.upper()`	Return uppercase text: "PYTHON".
`" hello ".strip()`	"hello": remove surrounding whitespace.
`text.replace("Py", "Jy")`	"Jython": replace matching text.
`"red,blue".split(",")`	["red", "blue"]: split text into a list.
`", ".join(["red", "blue"])`	"red, blue": join strings together.
`text.find("th")`	2: first matching index; -1 if absent.
`text.count("o")`	1: count occurrences.
`text.startswith("Py")`	True: the text starts with "Py".
`text.endswith("on")`	True: the text ends with "on".
`"Hi " + "Alex"`	Concatenate (join) strings: "Hi Alex".
`"ha" * 3`	Repeat a string: "hahaha".

Strings cannot be changed in place. Save a returned value to keep it: text = text.lower(). Use matching single or double quotes around text.

3.2 Working with lists

Create an ordered, changeable collection: nums = [10, 20, 30]. Each row below starts with that original list.

Python	Meaning or result
`nums[0]`	10: the first item.
`nums[-1]`	30: the last item.
`nums[1] = 99`	Replace an item: [10, 99, 30].
`nums.append(40)`	Add one item at the end: [10, 20, 30, 40].
`nums.extend([40, 50])`	Add several items: [10, 20, 30, 40, 50].
`nums.insert(1, 15)`	Insert before index 1: [10, 15, 20, 30].
`nums.remove(20)`	Remove the first matching value: [10, 30].
`last = nums.pop()`	Remove and return the last item: last is 30.
`del nums[0]`	Remove the item at index 0: [20, 30].
`len(nums)`	Number of items: 3.
`sum(nums)`	Total of the values: 60.
`min(nums)`	Smallest value: 10.
`max(nums)`	Largest value: 30.
`sorted(nums, reverse=True)`	Return a new sorted list: [30, 20, 10].
`nums.sort()`	Sort the original list in ascending order, in place.
`nums.reverse()`	Reverse the original list in place: [30, 20, 10].
`nums.count(20)`	Count occurrences of 20: 1.
`nums.index(20)`	Find the first index of 20: 1.
`copy = nums.copy()`	Make a shallow copy of the list.

append(), sort() and reverse() return None. Use nums.sort(), not nums = nums.sort(). remove() and index() raise ValueError if the value is missing.

4 Collections files and debugging

4.1 Dictionaries tuples and sets

Python	Meaning or result
`student = {"name": "Alex", "mark": 72}`	Dictionary: store values under keys.
`student["name"]`	Read the value "Alex" using its key.
`student["mark"] = 80`	Update a value; a new key would add an entry.
`student.get("age", 0)`	Return the value, or 0 if the key is absent.
`"mark" in student`	Test whether a key exists.
`student.items()`	Key/value pairs for a loop.
`point = (3, 4)`	Tuple: an ordered collection you cannot change.
`colours = {"red", "blue"}`	Set: unique values, with no positional indexing.
`colours.add("green")`	Add a value to a set.
`colours.discard("red")`	Remove a value if present; no error if absent.

4.2 Useful loop patterns

```
for key, value in student.items():
    print(key, value)
for position, item in enumerate(["red", "blue"], start=1):
    print(position, item)     # 1 red, then 2 blue
```

4.3 Read and write text files

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Hello\n")
with open("notes.txt", "r", encoding="utf-8") as file:
    contents = file.read()
print(contents)
```

Modes: "r" reads; "w" creates or overwrites; "a" appends. with closes the file automatically. A relative filename is resolved from the program's current working directory.

4.4 Handle invalid input

```
try:
    age = int(input("Age: "))
    print(f"Next year: {age + 1}")
except ValueError:
    print("Please enter a whole number.")
```

This catches invalid conversion once; use a loop if you want to ask again. Catch the specific error you expect.

4.5 Common errors and quick fixes

Python	Meaning or result
`SyntaxError`	Check colons, quotes, brackets and Python syntax.
`IndentationError`	Check indentation and use consistent spaces.
`NameError`	Check spelling and define variables before use.
`TypeError`	Check types: "3" + 2 fails; int("3") + 2 works.
`ValueError`	The type fits, but the value does not: int("hello").
`IndexError`	The sequence index is out of range.
`KeyError`	The dictionary key does not exist.
`ZeroDivisionError`	Check that a divisor is not zero.

Read the last line of the traceback first, then the referenced line in your code. Use print(value, type(value)) to inspect a value. Remember: = assigns; == compares.

5 Useful modules and turtle graphics

5.1 Import extra tools

Python	Meaning or result
`import random`	Load the random module.
`random.randint(1, 6)`	Random integer from 1 to 6, including both ends.
`random.choice(["red", "blue"])`	Choose one item from a non-empty sequence.
`random.random()`	Random float from 0.0 up to, but excluding, 1.0.
`random.shuffle(cards)`	Shuffle a list called cards in place; returns None.
`import math`	Load mathematical functions and constants.
`math.sqrt(25)`	Square root: 5.0.
`math.pi`	The constant π: approximately 3.14159.
`math.floor(3.8)`	Round down to an integer: 3.
`math.ceil(3.2)`	Round up to an integer: 4.
`import time`	Load the time module.
`time.sleep(1)`	Pause for one second.

5.2 Turtle drawing commands

Start with import turtle, then pen = turtle.Turtle(). Turtle opens a graphics window in a suitable desktop Python environment. Browser and notebook environments may need a turtle-compatible setup.

Python	**Meaning or result**
`pen.forward(100)`	Move forward by 100 units.
`pen.backward(50)`	Move backward by 50 units.
`pen.right(90)`	Turn clockwise by 90 degrees.
`pen.left(90)`	Turn anticlockwise by 90 degrees.
`pen.penup()`	Stop drawing while moving.
`pen.pendown()`	Resume drawing while moving.
`pen.goto(0, 0)`	Move to coordinates (x, y). The centre is (0, 0).
`pen.setheading(0)`	Face right. 90 = up; 180 = left; 270 = down.
`pen.pencolor("blue")`	Set the line colour.
`pen.pensize(3)`	Set the line thickness.
`pen.fillcolor("gold")`	Set the fill colour.
`pen.begin_fill()`	Call before drawing the shape to be filled.
`pen.end_fill()`	Call after drawing to fill the shape.
`pen.circle(50)`	Draw a circle of radius 50.
`pen.dot(10)`	Draw a filled dot of diameter 10.
`pen.speed(0)`	Fastest drawing. Speeds 1 to 10 are animated.
`pen.clear()`	Erase this turtle's drawing.
`pen.hideturtle()`	Hide the turtle pointer.
`turtle.done()`	Keep the window open; put at the end.

5.3 Complete example drawing a square

```
import turtle
pen = turtle.Turtle()
pen.pencolor("blue")
pen.pensize(3)
for side in range(4):
    pen.forward(100)
    pen.right(90)
turtle.done()
```

For a regular polygon with n sides, repeat n times and turn 360 / n degrees each time. Try n = 3, 5 or 6.